

Lab 5 – Convolution

EE252L/CE251L: Signals and Systems Lab

Dr. Basit Memon and Ms. Hira Mustafa

Name: _____

Objectives

By the end of this lab, you will be able to get familiarize with continuous and discrete time convolution, its relation with LTI Systems and its application on 1-D systems using MATLAB.

Instructions: After completion of each task get it marked by sharing your results and code with any of the **instructor**, your code must be self-explanatory i.e. use proper comments.
Attach rubrics at the end of your report.

1 Convolution

This lab is centered around another operation that can be performed on signals, called 'Convolution'. Convolution has a central role in signal processing too, and you'll discover the same in this lab. Given two continuous-time signals $u(t)$ and $v(t)$, the continuous-time convolution operation can be defined as:

$$y(t) = u(t) * v(t) = \int_{-\infty}^{\infty} u(\tau)v(t - \tau) d\tau, \quad (1)$$

Where the symbol $*$ denotes convolution, and it must not be confused with the multiplication symbol. The value of the convolution output at some time t is defined as the integral of the products of the two input signals, where one of the signals has been shifted in time by t and flipped.

Task 1 - Warm up

In your first task, you're provided a demo of continuous-time convolution (courtesy of Mathworks "Convolution in Digital Signal Processing" Courseware), and you'll play around with the inputs to get more familiar with the convolution operation.

- Download and open the `convolution_basics_1.mlx` file, located in the files section in folder Lab5 on LMS.
- Use the sliders and drop downs to set different parameters for the signals. Click Run to view the convolution process and its result. The demo provides some suggestions to try at the bottom too.
- Answer the following questions:
 - Does the order of inputs matter? Is convolution always commutative, i.e. is $u(t) * v(t) = v(t) * u(t)$?
 - Describe your general observations about the convolution output w compared to the inputs u and v ?

The convolution operation can similarly be described in discrete-time too. Given two discrete-time signals, $u[n]$ and $v[n]$, their convolution can be defined as:

$$w[n] = u[n] * v[n] = \sum_{k=-\infty}^{\infty} x[k]x[n - k].$$

A demo similar to continuous-time case is provided for discrete-time too. While the previous demo used a summation to approximate a continuous integral, discrete-time convolution can be exactly computed in MATLAB using `conv` command.

Task 2 - Warm up continues

- The demo for discrete-time convolution is located in the same file. Play around with the various parameters to enhance your understanding of discrete-time convolution.
- Answer the following questions:
 - How is the length of convolution output related to the length L of the two input signals?
 - What do you think will happen if the input sequences had different lengths? say, M and N , instead of L .

Can you use the `conv` command to compute continuous-time convolution? Yes; you can find out how in Appendix-A.

2 Why do we care about convolution?

In this section, we'll find out that the convolution operation provides a remarkable way to implement signal processing through linear-time invariant systems. We'll discover this by looking at a discrete-time system, shown in Figure 1, which computes the moving average of an input signal $x[n]$ to generate the corresponding output signal $y[n]$.

The moving average operation

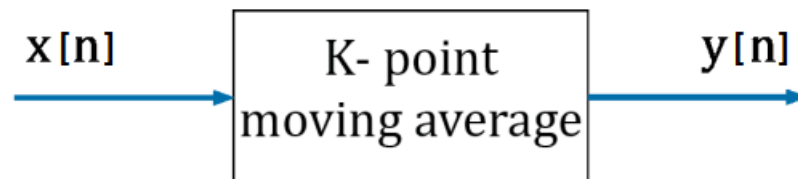


Figure 1: Moving Average Filter

The output of a K -point moving average system at any time n is simply the average (mean) of the input at the same time and some $K - 1$ neighboring values of the input. For example, the output of a five-point centered moving average system is defined as:

$$y[n] = \frac{1}{5} (x[n-2] + x[n-1] + x[n] + x[n+1] + x[n+2]) .$$

You may be surprised to know that this system acts as a simple low-pass filter on the data, i.e. it removes high frequency components from the signal or data. This is why the system is also known as a moving-average filter, as it removes unwanted noise. As the length of the filter increases, the smoothness of the output increases, whereas the sharp modulations in the data are made increasingly blunt.

Task 3: Experimenting with moving-average filter

In this task, you are going to implement a 5-point moving average filter. The MATLAB command `movmean` can be used for its implementation. Use MATLAB `help` to explore this command.

- (a) Find the output of this moving average filter for the input $x[n]$, where $x[n] = z[n] + \text{noise}[n]$ and $z[n] = \sin(0.3142 \cdot n)$. First generate a noisy signal using the command `randn` and add it to $z[n]$ to finally acquire $x[n]$, where $n = -20:20$. Store the output in a variable named `y1`.
- (b) Find the output of this moving average filter for a unit-impulse input $x[n] = \delta[n]$. Store the resulting output in the variable `h`.
- (c) Convolve $x[n]$ from (a) and $h[n]$ from (b), and store the output in `y2`.
- (d) Plot $x[n]$, $y1[n]$, $h[n]$, and $y2[n]$ in the same figure and comment on the differences/similarities between $y1$ and $y2$. **Preferred plotting approach:** Use plot command instead of stem, you will be able to observe the graphs clearly.

The observation that $y1[n] = y2[n]$ is not a coincidence, and will hold for any $x[n]$. This is because the moving average system is a linear-time invariant (LTI) system, and it is a property of LTI systems that they are completely specified by their impulse responses, which is usually denoted by $h[n]$. More specifically, if the impulse response of an LTI system is known, one can compute the system output for any input signal. We now present one of most important topics in signals and systems theory:

“The response (or output) of an LTI system to any input signal is computed by the convolution of the input signal with the impulse response of the system, i.e.

$$y[n] = x[n] * h[n],$$

where $h[n]$ is the impulse response of the system, $x[n]$ is any input, and $y[n]$ is the corresponding output.”

What is the impulse response? The impulse response of an LTI discrete-time system is the output of the system when the unit impulse is applied as input, as was the case in (b) of the previous task. Even though the unit impulse input consists of one nonzero term, the impulse response signal $h[n]$ usually consists of more than one nonzero elements. The explanation is that the system is dynamic (with memory); that is, the system responds over various time instances to the input applied at $n = 0$.

Task 4: Is moving-average filter an LTI system?

Prove that the moving-average filter is a linear time-invariant system analytically.

In the next task, you will see a powerful application of the output principle of an LTI system using convolution. You would have surely noticed that the same audio (you speaking or music) played in different physical environments sounds different. This is called reverberation, an audio

effect, resulting from the interaction of sound waves with the environment. By using our output principle, we can artificially introduce reverberation to make our sound appear to be playing in a different environment than the one in which it was recorded. For example, in the following task you will make a ringtone recorded in a recording studio (anechoic chamber) sound as if it were playing in a church balcony in Nashville.

Task 5: Response of LTI systems

Pass a sine wave through various (at least two) LTI systems and show that output is sine wave of same frequency.

1. **System Implementation:** Generate a sine wave and process it through two distinct LTI systems using a `Moving Average` filter or `conv` function.
2. **Visualization:** Plot the input and both output signals on the same axes. Document the observed changes in signal amplitude and the horizontal shift (phase) introduced by each system.
3. **Frequency Verification:** Use the `findpeaks` command to extract the time locations of the output peaks. Determine the time period (T) between consecutive peaks and calculate the frequency $f = 1/T$ to mathematically prove it matches the input frequency.

Task 6: Reverberation

Let's import a recording of an impulse sound signal reverberated from a church balcony^a. This will be the impulse response of the church modeled as an LTI system.

```
|| [h, Fh] = audioread("1st_baptist_nashville_balcony.wav");  
|| disp(size(h))
```

Notice (by observing the size) that this audio has two channels (left and right). It is a stereophonic sound. Let's play this audio

```
|| soundsc(h,Fh) % Play the sound  
|| pause(length(h)/Fh) % Wait for audio to finish playing
```

Let's import the ringtone to which we want to apply the same reverberation effect. Notice that this file has just one channel. It is a monophonic recording. Let's play it.

```
|| [x, Fx] = audioread("ringtone.wav"); % Read input  
|| disp(size(x))  
|| soundsc(x,Fx) % Play the sound  
|| pause(length(x)/Fx) % Wait for audio to finish playing  
  
|| h1 = h(:,1); % Extract the first channel from the impulse response  
|| %In simple words extracting all the rows of first column  
|| h2 = h(:,2); % Extract the second channel from the impulse response  
|| %ISimilarly extracting all the rows of second column of h
```

- Using `conv` command, compute convolution of `h1` and `h2` separately with `x`, i.e. $y_1 = h_1 * x$ and $y_2 = h_2 * x$. Concatenate the two outputs, $y = [y_1, y_2]$. Play the resulting sound `y` using `soundsc` and sampling frequency `Fx`.
- Plot the audio signals. And state your observations.
- Apply this effect to your own sound files! You can find impulse responses of more interesting places at https://www.openair.hosted.york.ac.uk/?page_id=36 and some anechoic recordings at https://www.openair.hosted.york.ac.uk/?page_id=310. State your observations.

^aThe audio file `1st_baptist_nashville_balcony.wav` by Adam Townsell is licensed under CC-BY and was obtained from <https://openairlib.net/>.

Post Lab - Task 7: Write your own convolution function

To make sure that you understand discrete-time convolution, you are now to write your own MATLAB function `w = discrete_convo(u,v)` to compute discrete-time convolution of two input signals `u` and `v`. Your function should not use MATLAB's built-in command `conv`.

Test your function by convolving $h[n] = [1, 2, 1], -1 \leq n \leq 1$ with $x[n] = [1, 2, 3, 4, 5], 0 \leq n \leq 4$. Convolve the same two signals using MATLAB's built-in `conv`. Compare the two outputs by plotting them in the same figure along with the inputs. Set `axis([-2 5 0 8])`.

Hint: The process that has to be followed to compute the convolution sum is as follows. First, the two input signals are plotted on the k -axis. Then, one of the two signals is reflected about the amplitude axis. The convolution output at any time n is computed by shifting the reflected signal along k -axis by the value n , finding the product of overlapping values of the two signals, and computing the sum of product terms, according to the convolution sum

$$y[n] = x[n] * h[n] = \sum_{k=-\infty}^{\infty} x[k]h[n-k].$$

A Continuous-time Convolution using the `conv` command

The computational process followed in demo approximates the analytical way of deriving the convolution between two signals. In MATLAB, the command `conv` allows the direct computation of the convolution between two signals.

There are four rules that have to be applied for successfully computing the convolution between two continuous-time signals.

First rule: The two signals (input and impulse response) should be defined on the same time interval. Say, both signals are defined on the time interval $a \leq t \leq b$, where $t = a$ is the first time instance that at least one of the signals $x(t)$ or $h(t)$ is not zero and $t = b$ is the last time instance that at least one of the two signals is non-zero.

Example:

$$h(t) = \begin{cases} 1-t & 0 \leq t \leq 1 \\ 0 & \text{elsewhere} \end{cases} \quad (2)$$

$$x(t) = \begin{cases} 1 & 0 \leq t \leq 2 \\ 0 & \text{elsewhere} \end{cases} \quad (3)$$

Making the intervals same by writing $h(t)$ as:

$$h(t) = \begin{cases} 1-t & 0 \leq t \leq 1 \\ 0 & 1 > t \geq 2 \end{cases} \quad (4)$$

Second rule: When a signal consists of multiple parts, the time intervals in which each part is defined must not overlap.

$$h(t) = \begin{cases} 1 - t & 0 \leq t \leq 1 \\ 0 & 1 > t \geq 2 \end{cases} \quad (5)$$

Notice that the equality at $t=1$ is placed only at the upper part. In MATLAB, it will be defined as:

```
dt=0.1;
t1=0:dt:1;
t2=1+dt : dt : 2; %time does not overlap it starts from (1+dt = 1.1 second)
h1=1-t1;
h2=zeros(1,length(t2));
h=[h1 h2];
```

Third rule: The output of the `conv` command has to be multiplied with the time step used in the definition of the input and impulse response signals, in order to correctly compute the output of the system. This rule emerges from the fact that the convolution integral is being approximated by discrete sum in MATLAB. It is written as: $y = \text{conv}(x, h) * dt$.

The only thing left to do is to plot the output. However, the number of elements of the output vector y is not equal to the number of elements of the vectors x or h . The precise relationship is: $\text{length}(y) = \text{length}(x) + \text{length}(h) - 1$.

Fourth rule: The output of the system is plotted in the double time interval of the one in which the input and impulse response signals are defined. i.e for the above stated example $t_y = 0:dt:4$.